

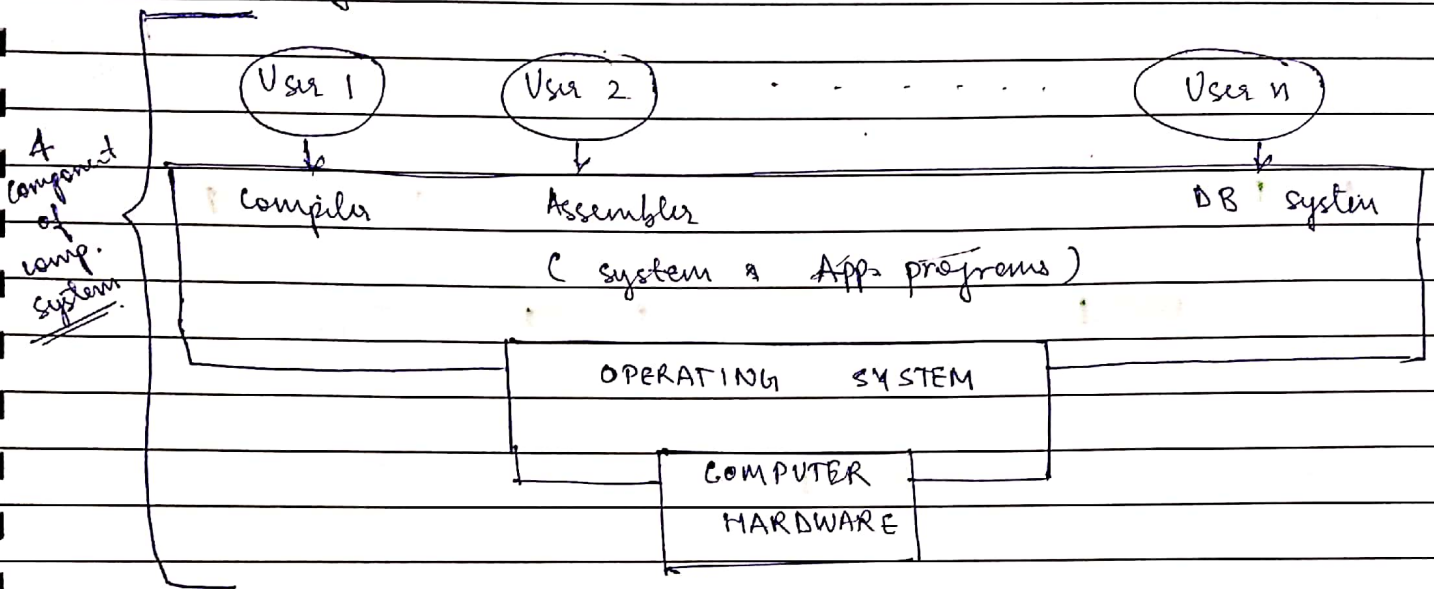
- when to remove comment!

... hand to another formula

cell already

OPERATING SYSTEM

- computer has resources (hardware & software) → to complete a task.
eg: i/o devices, memory, file storage space, CPU etc.
- OS acts as a manager of above resources and allocates them to specific programs & users as necessary for their task
- **Resource manager**: manages resource of computer system internally.



FUNCTIONS OF OS

- Boots the computer
- Performs basic computer tasks (managing various peripheral devices)
 mouse, keyboard etc.
- Provides user interface (GUI, command line)
- Handles system resources (comp. memory & share of CPU time)
 by various apps / peripheral devices.
- File management
 (How OS manages, stores, retrieves & saves data)
- Error handling → takes preventive measures when reqd to avoid errors

OS PROCESSES

Programme in execution: Process - 'active' entity
 (working prog - ready for i/o) Programme - 'passive' entity

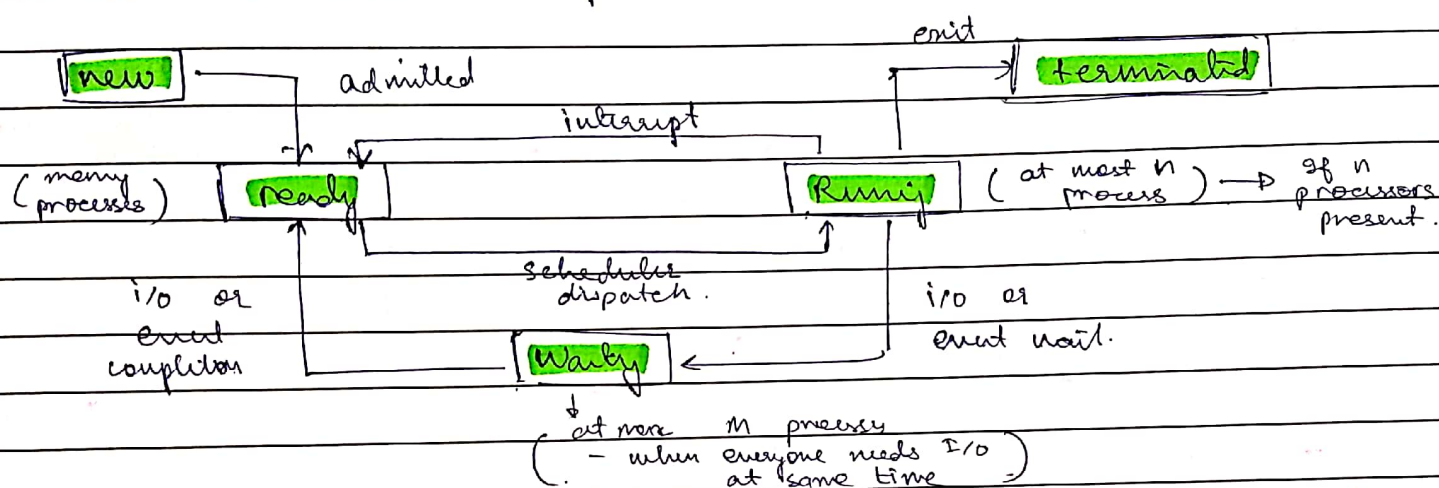
memory allocated before prog.

Process memory divided into 4 sections:

- Text section** - compiled prog. code when prog. is launched
- Data section** - global & static variables, allocated & initialised prior to execution
- Heap** - for dynamic memory allocation, managed via calls to new, delete, malloc, free etc.
- Stack** - local variables (space on stack reserved for local variables)
 memory made when needed

PROCESS STATE → defines current state of process.

- 1) New — process in stage of being created
- 2) Ready — all resources available that it needs to run, but CPU not currently working on this process's instructions
- 3) Running — CPU is working on this process's instruction.
- 4) Waiting — Process cannot run at the moment, because waiting for some resource to become available / some event to occur.
- 5) Terminated — process completed



MODES OF OS

- ① **Kernel mode** → direct access to memory & hardware
→ whole system can halt even in minor faults
- ② **User mode** → executes programmes
→ programme will halt.
→ OS in user mode (safe state).

→ each process represented in CPU by PCB.

PROCESS CONTROL BLOCK (per each process)

→ Data structure — stores information about a process

- **Process state** — running, waiting etc.
- **Process ID**
- **CPU registers** & **Program counter** (pointer holds pointer address of next instruction to be executed for that process)
- **CPU scheduling information** — priority info, pointers to scheduling queues
- **Memory management info** — page table, segment table
- **Accounting info** — user & kernel CPU time consumed, account nos
- **I/O status info** — devices allocated, open file tables etc.

PROCESS SCHEDULING QUEUES

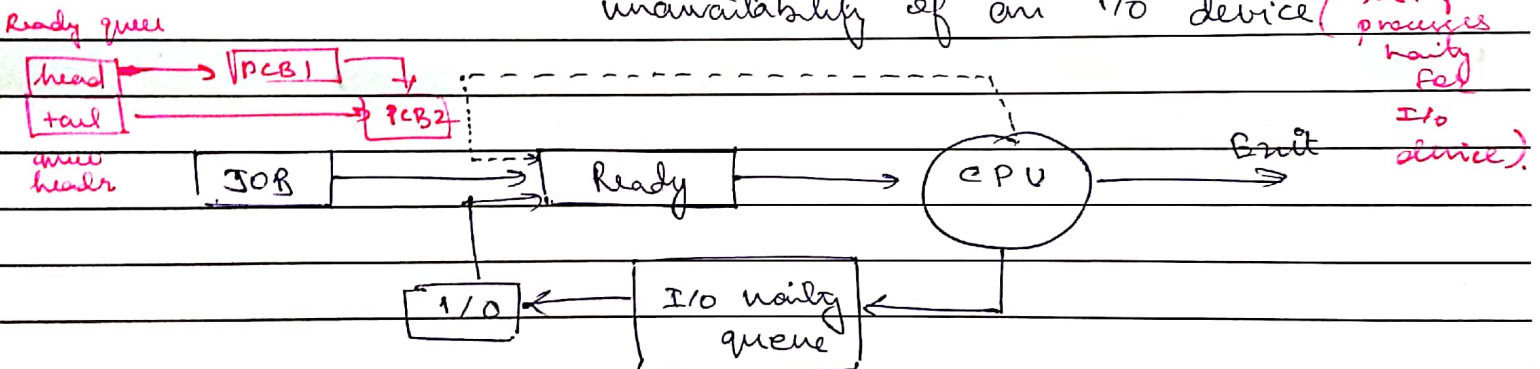
activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on basis of particular strategy.

- OS maintains all PCB's in process scheduling queues
- ~~OS~~ ~~now~~ separate queue for each process state and PCB's of processes in same state $\rightarrow$ placed in same queue
- state of process changed $\rightarrow$ PCB unlinked from its current queue & moved to its new state queue.

① Job queue - keeps all processes in system

② Ready $\downarrow$ - processes in ready state - residing in main memory, ready & waiting to execute
linked list - queue head, tails

③ Device - processes which are blocked due to unavailability of an i/o device (*list of processes waiting for I/O device*)



SCHEDULERS IN OS

① Long term scheduler : processes to shift from new $\rightarrow$ ready.
- controls degree of multiprogramming. (*No of processes in memory*)
- ~~no~~ avg (processes created) = avg (processes shifted to ready).
If degree of multiprogramming stable! $\rightarrow$ rate of incoming = rate of departure.

② Short term / CPU scheduler : ready $\rightarrow$ running
- selects processes to shift to running state $\rightarrow$
this scheduler allocates memory to running process

③ Medium term : swap in - swap out (*remove processes*)
process wants i/o $\rightarrow$ swap in ready processes

PROCESS SCHEDULING ALGO

- arrival time — arrives in ready queue
- completion time — completes execution
- Burst — ~~(completion - arrival)~~ CPU execution time
- Turn around — (completion - arrival) $\rightarrow$ total time
- wait time — (Turn around - burst)

Throughput — Total no. of processes completed per unit time

Non-preemptive — once process enters running state, cannot be preempted until it completes its allotted time.

Pre-emptive — based on priority, scheduler may preempt a low priority running process anytime when a high " process enters ready queue.

OBJECTIVES OF PROCESS SCHEDULING ALGO

- Max CPU utilization (keep CPU as busy as possible)
- Fair allocation of CPU
- Max throughput
- Min. turnaround time
- Min. wait time
- Min. response time

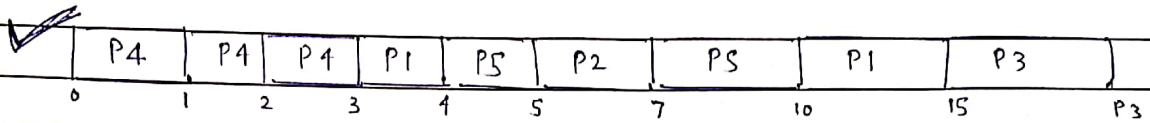
- ① Priority scheduling $\rightarrow$ max priority process completed first.
- ~~② Round Robin~~ $\rightarrow$ equal priority - (FCFS)
- $\rightarrow$ priority based on memory requirement, time requirement, etc.

- ② Round Robin $\rightarrow$ Fixed time (quantum) for each process
- $\rightarrow$ once process executed for given time — it is preempted & other process executed
- $\rightarrow$ context switching used to save state of preempted process
- $\rightarrow$ STARVATION avoided.

③ Pre-emptive shortest job first

pre-empted when process w/ shorter burst time arrives

Process	Burst	Arrival
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4



Waiting time :

P4	P1	P5	P2	P3
= 0	= 3 - 2 + 10 - 4	= 7 - 5	= 0	= 15 - 1
	= 7	= 2		= 14
= <u>4.6 ms</u>				

PROCESS SYNCHRONISATION AND RACE CONDITION

- ① Independent process - execution of one process does not affect execution of other processes
- ② cooperative - execution of one process affects the execution of other processes

Process synchronisation problem arises in cooperative process because resources are shared in cooperative processes.

Process syn. means sharing system resources by processes in such a way that, concurrent access to shared data is handled, thereby minimising chance of inconsistent data. Maintaining data consistency demands mechanisms to ensure synchronized execution of cooperating processes.

RACE CONDITION: multiple processes concurrently read and write to a shared memory location & result depends on order of execution.

eg of race condition:

count ++ ;

r1 = count.

r1 = r1 + 1.

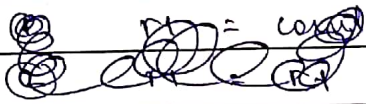
count = r1.

count -- ;

r2 = count

r2 = r2 - 1

count = r2.



let count = 5.

① count ++ , ② count --.

1 → r1 = count = 5

2 → r1 = r1 + 1 = 6

3 → r2 = count = 5

4 → r2 = r2 - 1 = 4.

5 → count = r1 = 6.

6 → count = r2 = 4 ⇒ X.

CRITICAL EXECUTION SECTION :

- piece of code that accesses a shared resource (data structure or device) that must not be concurrently accessed by more than one thread of execution.
- GOAL ⇒ provide mechanism by which only one instance of critical section is executing for a particular shared resource.

(difficult to detect critical section bugs).

do {

entry section

critical section.

exit section

remainder section

} while (TRUE);

CRITICAL SECTION SOLUTIONS

- critical section must enforce all 3 of foll. rules.

- ① **Mutual Exclusion** — if P_i is executing in critical section, no other process can execute their CS.
- ② **Progress** — no process in CS, & there exist some processes that wish to enter the critical section, then the selection of processes that will enter CS next, cannot be postponed indefinitely.
- ③ **Bounded waiting** — Bound must exist on no. of times that other processes are allowed to enter their CS after a process has made request to enter its CS & before that request is granted.

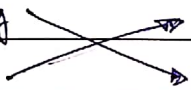
PETERSON'S SOLUTION

— requires 2 processes to share 2 data items.

```

whose turn to enter?
    int turn;
    bool flag[2]
}
flag[0] = T ⇒ P0 ready to enter CS
turn can be 0, 1.

is process ready?
do {
    flag[i] = T;
    turn = i;
    // WAIT.
    while ( flag[j] && turn == j );
    CRITICAL SECTION
    flag[i] = F;
    REMAINDER.
} while (true)
  
```

- ✓ Mutual exclusion ($turn = 0$ or $turn = 1$)
- ✓ Bounded waiting (after CS, $flag[i] = F$)
- ✓ Progress.  only 2 processes, each process has to wait for 1 process, to enter its CS.

• if P_0 is in CS, and P_1 wants to enter, it can context switch P_0 , but cannot run in CS $\Rightarrow$ it will have to switch back, else deadlock.

SEMAPHORES → integer variable.

changed using:

① wait(s) { while (s == 0);
s--; } ② signal(s) { s++; }

• can be only as binary (0-1).

• Binary semaphores → aka mutex locks, because provides mutual exclusion.

• Counting semaphores → used to control access to a val = no. of instances of resource. { given resource contains of a finite no. of instance.
val > 0 ⇒ free resource available, process can enter.

⇒ Critical section of n processes:

semaphore mutex; // initially mutex = 1.

Process P_i:

do {

wait (mutex);

CRITICAL.

signal (mutex);

REMAINDER;

} while (1);

// wastes CPU cycle.

while (s == 0);

Busy waiting problem → as processes may wait in CPU,

↓ solution (remove waiting processes)
(s == 0)

holding memory & continuously checks CS for it's chance.

list of waiting processes

typedef struct { int value;

struct process * list; } semaphore;

wait (semaphore * s) { s → val --;

Storing as processes which were waiting in a semaphore,

Remove process from CPU

{ if (s → val < 0) { add this process to s → list; block(); }

}

Signal (semaphore * s) { s → val ++;

and taking them in memory.

picks one process from list

{ if (s → val <= 0) {

Pickup process P from s → list; wakeup (P); }

PRODUCER CONSUMER PROBLEM.

- Buffer $\rightarrow$ produce, consume.

Producer :

```
while (true) {
    // produce an item in next produced
    while (counter == BUFFER_SIZE); // wait
    buffer[in] = next produced;
    in = (in + 1) % Buffer - size;
    counter++;
}
```

Consumer

```
while (true) {  
    while (center == 0); // wait.  
    next consumed = buffer[out];  
    out = (out + 1) % buffer-size;  
    center--;  
    // consume the item in next  
    consumed
```

⇒ at a time it should run only for one process
⇒ critical section solution.

• shared :

semaphore :	$\text{full} = 0$,	$\text{empty} = n$, ↓ <u>buffer-size</u>	$\text{mutex} = 1$ ↓ either producer runs or consumer
-------------	---------------------	---	--

Producer : Consumer

Producers :

do 2.

— produce an item in next p.

`wait (empty);` $\rightarrow$ `empty == 0` $\Rightarrow$ WAIT

`wait (msec);` → only one process in CS

- critical: add nrotp to buffer

signal (mutex); $\rightarrow$

signal (full); $\rightarrow$ 7 full.

3 white (1)

Consumer

 $\partial \Sigma$

```
wait (full);
```

```
wait(mutex);
```

- remove an item from buffer.

signal (mutex);

```
signal (empty);
```

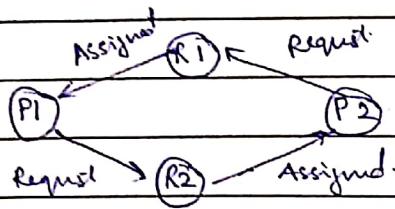
- consume item;

3 white (1)

DEADLOCK

Process :

① Requests the resource → ② Uses the resource → ③ Release the resource



All of (simultaneously) these conditions ⇒ Deadlock.

① Mutual exclusion

② Hold and wait

③ No preemption

④ Circular wait

① Mutual exclusion → at least one non-shareable resource.

② Hold & wait → process is holding one resource, & requesting another resource held by other process

③ No preemption → can obtain a requested resource, only when other process releases it.

④ Circular wait → cycle (P1 → R1 → P2 → R2)

METHODS TO HANDLE DEADLOCK

① Deadlock prevention / avoidance → make sure deadlock never occurs in our system

② Deadlock detection & Recovery.

③ Ignore and reboot → Windows, UNIX.

DEADLOCK PREVENTION :

→ prevent any 1 of 4 conditions.

① Mutual exclusion → make resource ~~by~~ used by 2/more processes
⇒ not possible for non shareable resource. eg: printers ⇒ cannot be avoided

② Hold and wait →

→ assign all resources a process needs initially.

→ not possible → ① we do not know beforehand what is needed
② may lead to starvation

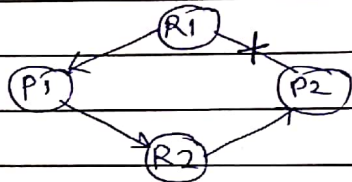
③ Pre-emption → not possible → may lead to inconsistency
eg: P1 halfway through using printer → now P2 demands ⇒ cannot be preempted

④ Circular wait → can be avoided

Avoiding circular wait :

- assign priority to each resource.
- any process can request resources only of higher priority.

eg: $P(R1) = 1$ $P(R2) = 3$ $P(R3) = 5$.



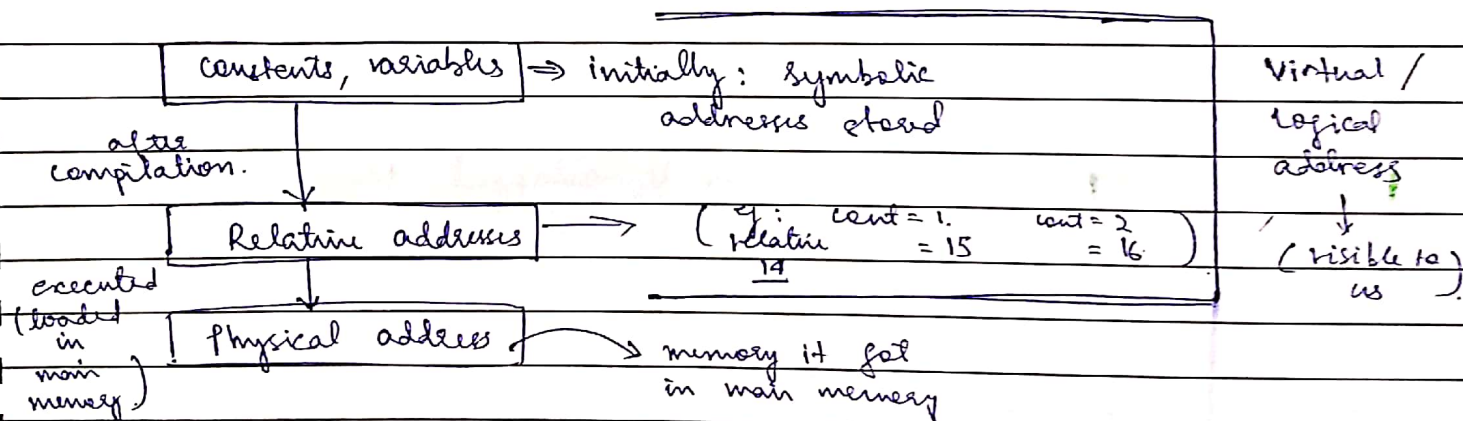
BANKER'S ALGO → FOR AVOIDANCE

↳ deadlock avoidance & resource allocation algo → tests for safety, by simulating the allocation of maximum possible amounts of all resources.

- ↳ ① Safety algo. → checks if system is in safe state
- ↳ ② Resource - request algo. → checks if resource can be fulfilled or not.

MEMORY MANAGEMENT :

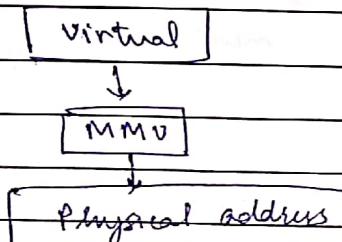
→ Process Address space : — set of logical addresses that a process addresses in its code.

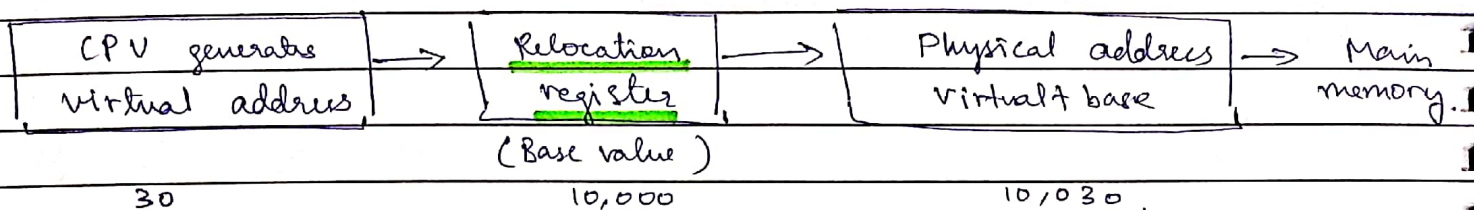


MMU (memory management unit)

before execution, virtual address $\neq$ physical address

after execution,





STATIC AND DYNAMIC LOADING :

- A programme can have dependency on other modules

① Static loading → compiler stores all the dependencies, beforehand and stores the overall code.
⇒ whole dependency is guaranteed at time of compilation

② Dynamic loading → stores references of dependencies before compilation
↓
loading in memory → when prog. is run & dependency is required then they are loaded in memory (during runtime)

Static and Dynamic linking → (done by linker)

linking done before compilation.

references stored, linking done during compile time.

SWAPPING - process can be swapped temporarily out of main memory (or more) to secondary storage (disk) and make that memory available to other processes.

- at some later, system swaps back the process from sec. storage to main memory.
- performance is usually affected, but, helps in running multiple & big processes in parallel
↓ aka

technique for memory compaction

⇒ Total time taken by swapping process: time to move entire process to sec. disk (+) copy process back to

(secondary) memory $\oplus$ time process takes to regain main memory (main)

2 transfers (main $\rightarrow$ sec, sec $\rightarrow$ main)

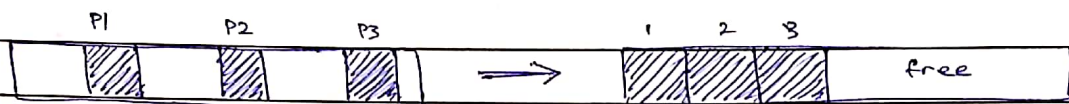
$\oplus$ ~~other~~ other overhead, where process compiles to regain main memory.

FRAGMENTATION

① External - memory cannot be allocated because, not continuous.

$\downarrow$ SOLUTION.

Requires dynamic allocation. $\rightarrow$ compaction $\rightarrow$ combine previously allocated memories, so now huge continuous chunk is free.
(so memory can be changed)

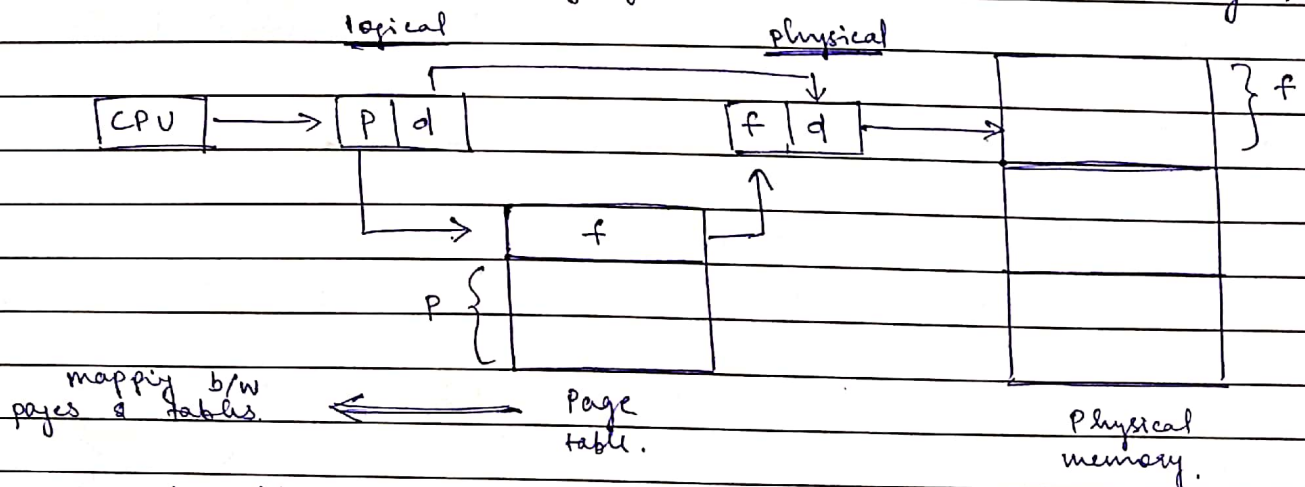


② Internal - extra memory allocated, than actually required. $\rightarrow$ other processes cannot be allocated

$\downarrow$ SOLUTION

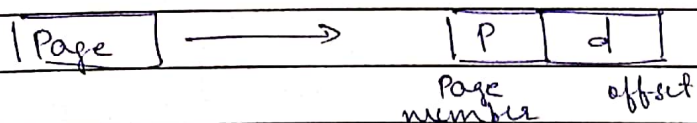
Allocate memory judiciously.

PAGING $\rightarrow$ to avoid external fragmentation, allow you store non-continuously.



logical address $\rightarrow$ no of page $\{$ Logical address space divided in pages
physical address $\rightarrow$ no of frames $\{$ PAs divided into frames.

Page size = frame size



$\text{LAS size} \rightarrow 2^m$
 $\text{Page size} \rightarrow 2^n$

$\frac{2^m}{2^n} = 2^{m-n}$: no of pages in LAS.

$\downarrow$
 offset: n size.

	0	a
P_0	1	b
	2	c
	3	d
P_1	4	e
	5	f

logical memory

Page no.	Frame no.
0	2
1	0

Page table

	6	d
	1	e
	2	f
	3	
	4	
	5	
	6	a
	7	b
	8	c

F_0

F_1

F_2

Physical memory

Physical = (Frame no) * (frame size) + offset

(b) $7 = (2) \times (3) + 1 = 7$

(c) $1 = (0) \times (3) + 1 = 1$

Internal fragmentation possible in paging

Calculating internal fragmentation.

Page size = 2048 bytes

Process size = 72,766 bytes $\Rightarrow$ 15 pages + 1086 bytes

$\text{internal fragmentation} = 2048 - 1086 = 962 \text{ bytes}$
 $\leftarrow$ 16 pages allocated

Worst case fragmentation : 1 frame $\rightarrow$ 1 byte.

on average : $1/2$ frame size.

$\Rightarrow$ Small frame size desirable.

ASSOCIATIVE MEMORY

→ Translation lookaside buffers.

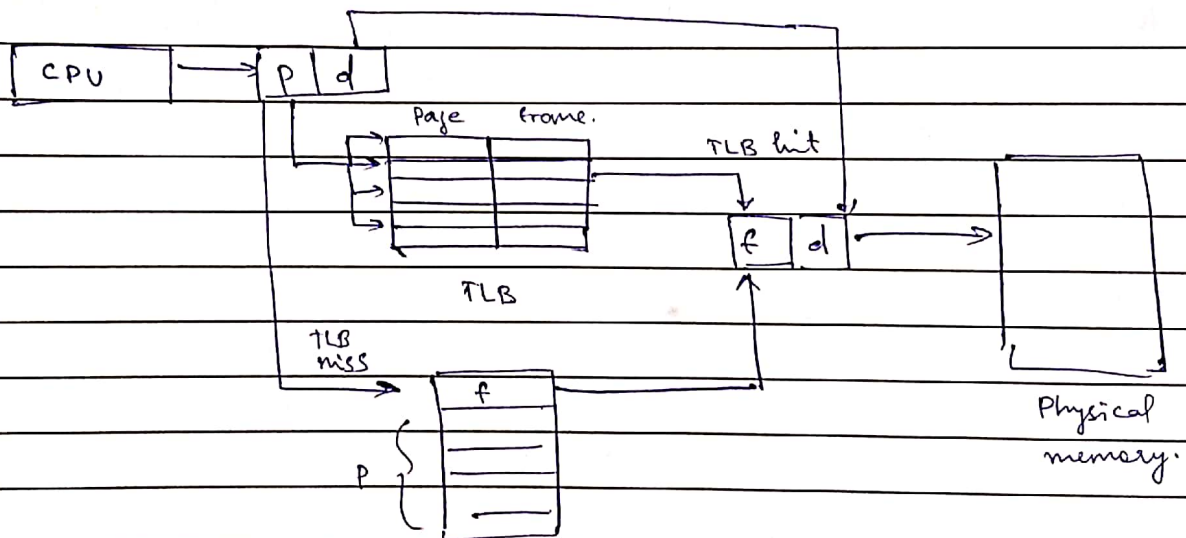
- PT kept in main memory.
- PTBA points to PT.
- PT length register (PTLR) indicates size of page table → can be huge.
- So here, every data / instruction access requires two memory accesses → ① access page table
can be reduced. ↓ ② data / instruction in frame.
- TLB's → aka associative memory.

→ special fast-lookup hardware cache
↓ parallel search.

• Address translation (p, d)

if p is in associative ~~memory~~ register, get frame # out, otherwise get frame # from page table in memory.

Page #	frame #



• Effective Access time.

eg: Hit ratio (% of time a page no is found in associative memory)

• $\alpha = 80\%$, 100 ns for memory access

$$\Rightarrow EAT = (0.80 \times 100) + (0.20 \times 200) = 120 \text{ ns}$$

(TLB hit 1 lookup to store in main) (TLB miss 2 lookup (page in main + p access))

• $\alpha = 99\%$, 100 ns for memory access

$$\Rightarrow EAT = (0.99 \times 100) + (0.01 \times 200) = 101 \text{ ns}$$

Page size
was fixed.

SEGMENTATION

- each job $\rightarrow$ divided into segments (diff. sizes)
 $\downarrow$
one for each module, that contains pieces that perform related functions.
- each segment has different address (LAS).
- Process executed $\rightarrow$ corresponding segments loaded non contiguously
 $\rightarrow$ every segment itself loaded in contiguous block of available memory.
- segment contains - program's main functions
- utility functions, data structures etc.
- OS maintains a segment map table for every process, and a list of free memory blocks along with
 - segment no
 - their size
 - corresponding memory location in main memory.
- for each segment, table stores:
 - starting address of segment
 - length of the segment
- reference to memory location = value identifying a segment + offset

PROBLEM

External fragmentation

$\rightarrow$ Process removed $\Rightarrow$ 200 bytes free in b/w,
new process = 100 bytes, cannot be allocated $\Rightarrow$ because
one process allocated contiguously.

- no internal fragmentation because variable sized segments.

VIRTUAL MEMORY

- can be implemented using both h/w and s/w.
- adds extra (virtual memory) to main memory.
 $\hookrightarrow$ system appears to have more memory than what actually exists.
- $\rightarrow$ VM: layer of memory addresses (virtual addresses),
that map to physical addresses
- $\rightarrow$ OS manages $\rightarrow$ virtual address space and
 $\hookrightarrow$ assignment of real memory to VM (mapping).

Working of VM:

- 1) Comp. has limited amount of static RAM.
- 2) When program executed, $\rightarrow$ an instance of prog loaded into RAM
(allocating memory for instructions to execute)
- 3) When prog. demands more RAM, then it'll be allocated to VM. $\rightarrow$ prevents the prog from lacking necessary RAM to execute.
- 4) VM is actually memory of hard disk & it is then mapped into physical memory.

Virtual memory is implemented using,

DEMAND PAGING $\rightarrow$ type of swapping done in virtual memory systems.

- aka lazy evaluation because
- Only demanded pages of memory are swapped from secondary storage (disk space) to main memory.
(in pure swapping, all the pages for a process were swapped).
- data copied from disk to RAM, when demanded by some programme.

Demand Paging working:

- based on page table (maps ~~logical~~ log. memory to physical memory)
- PT uses bitwise operator to mark if page valid/invalid
- Valid: currently resides in memory (main)
- Invalid: currently resides in secondary memory.
- When a process tries accessing a page,

① Attempt to access page

② exists in main memory
 $\Rightarrow$ Valid, page processing instruction continues as normal

③ Invalid $\rightarrow$ Page fault trap
④ checks if it is a valid reference to a location in secondary memory.

⑤ Does not exist in secondary
 $\Rightarrow$ TERMINATED (illegal memory access)

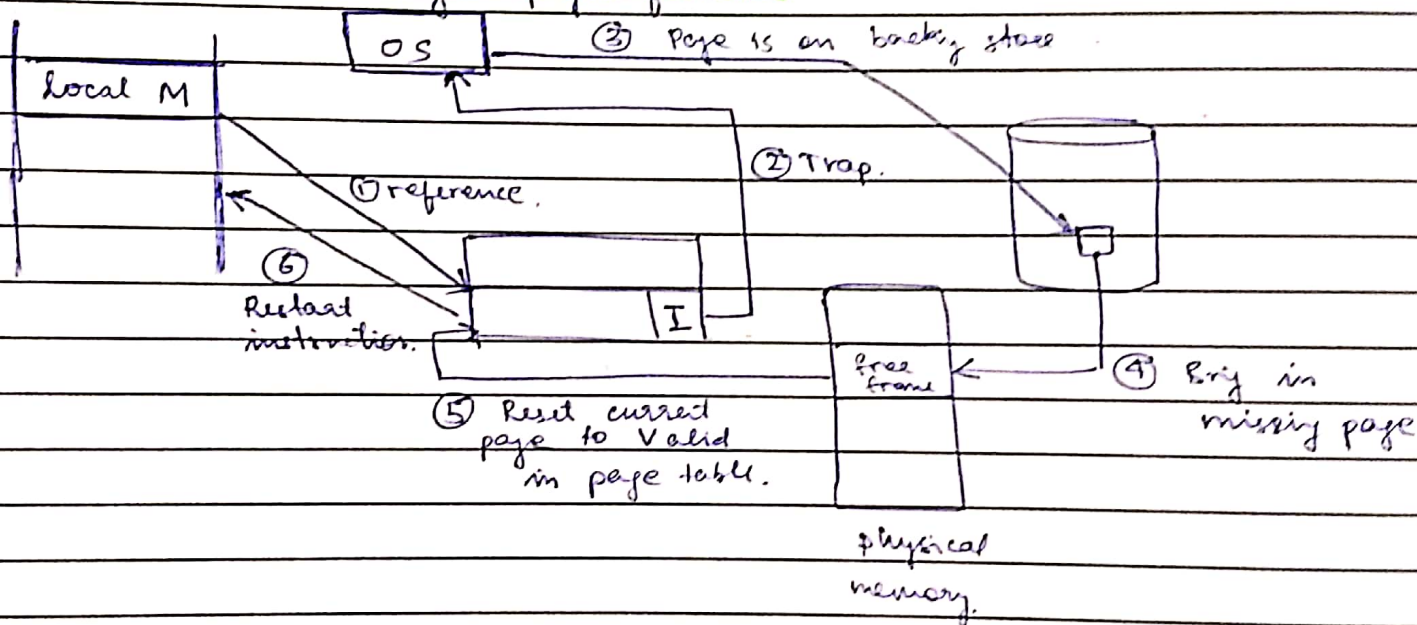
⑥ disk opⁿ to read page into main memory.
⑦ instruction interrupted by OS trap, now restarted.

- disk operation** : page table edited and current page is loaded, so on restart, it will be found in PT, a fault will not occur

o	a	page frame.	Valid-Invalid bit.	o		
1	b	0	4	V	4	A
2	c	1		I	5	
3	d	2	6	V	6	C
4	e	3		I		
		4		I		

Page Table physical memory Virtual memory.

Steps to handling page fault



PAGE REPLACEMENT ALGOS

- Page fault** : page exists in VM, not in physical memory.
 → to remove page fault → page replacement algo
 Replaces pages of physical memory, w/ required page in VM
- Reference string** : sequences of addresses which we have referred.

eg: (100 size pages) : addresses we refer: 112, 224, 336, 612, 86
 pages referred : 1, 2, 3, 6, 0
 reference string.

① FIFO (greedy).

0	4	2	0	2	1	6	4	0	1	0	3	1	2	1
2	0		M	M	M	M	M	M			M	M	M	
1	3													
6	1													

Fault rate = $\frac{\text{Miss}}{\text{Total}} = \frac{9}{12} = 0.75$

② Optimal Page replacement

- replace w page that'll be used last in forward string

0	3	0	2	1	6	1	0	1	0	3	1	2	1
2		M	M	M	M	M				M			
1													
6	4												

Fault rate = $\frac{\text{Miss}}{\text{Total}} = \frac{6}{12} = 0.5$

→ not practically possible → don't know page ka string.

③ Least Recently used (LRU)

0	4	2	0	2	1	6	4	0	1	0	3	1	2	1
2	0		M	M	M	M	M	M			M		M	
1														
6	3													

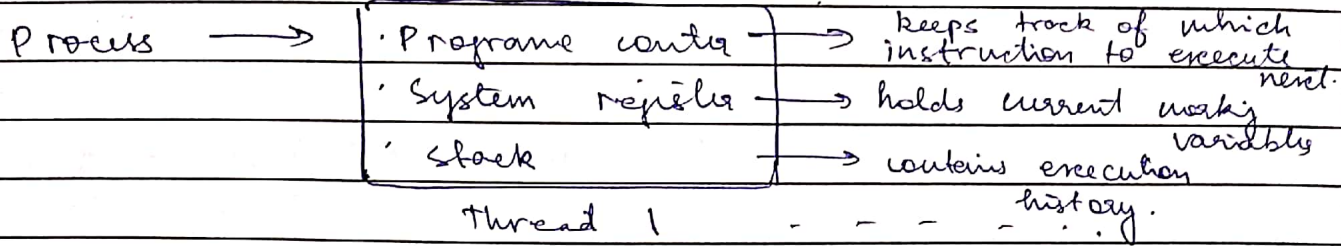
Fault rate = $\frac{8}{12} = 0.66$

④ Least frequently used.

⑤ Most frequently used

THREADS

- A process is divided into threads



- Threads share among themselves:

- common code segment, files

- one thread alters a code → ~~all~~ visible to all other threads

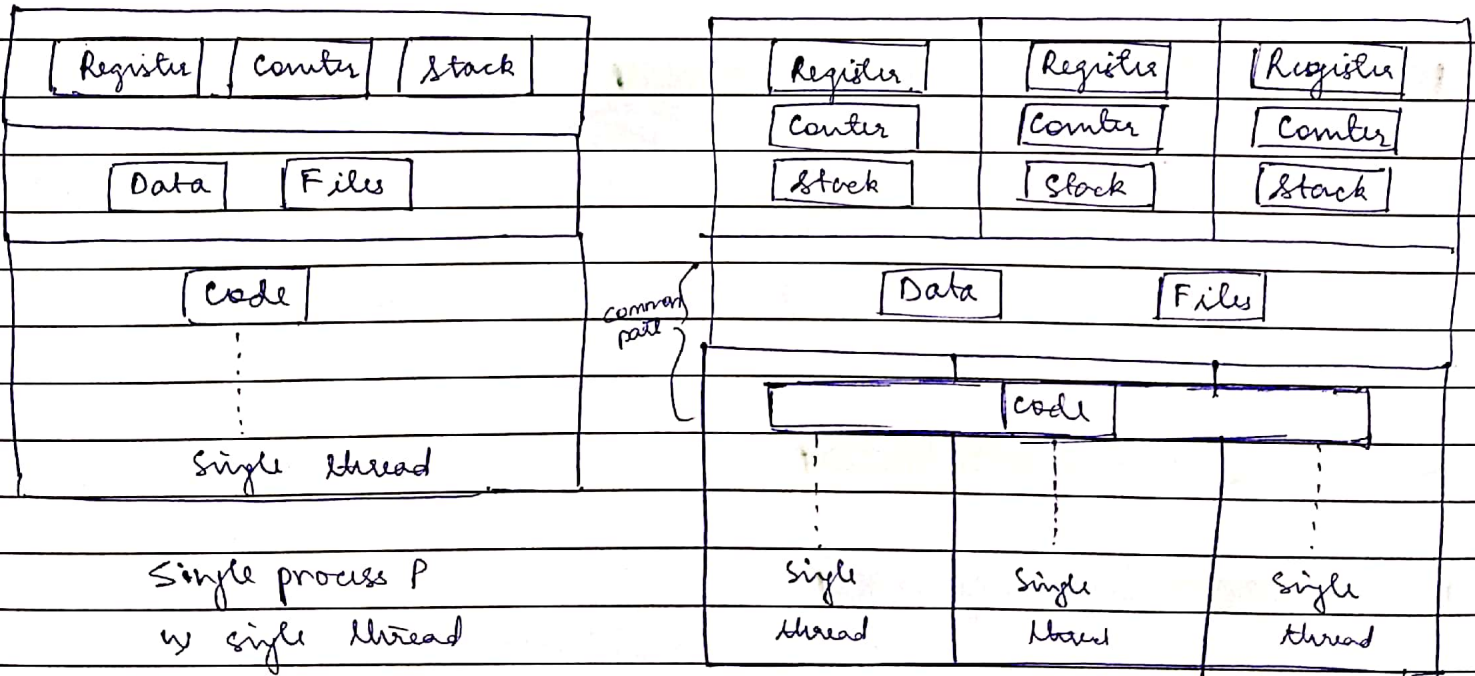
improved performance ← (PARALLELISM) ←

- aka - light weight process

- each thread belongs to exactly one process. → no thread can exist outside a process

→ used in implementing network servers and web servers

→ Each thread represents a separate flow of control



Single process P
w/ three threads.

(multithreading)

Difference b/w Process and thread

PROCESS

THREAD

- | | |
|---|--|
| ① Heavy weight, resource intensive | ① light weight, takes less resources than process |
| ② Process switching needs interaction w/ OS (system calls). | ② thread switching does not need interaction w/ OS |
| ③ multi process environment - each process executes <u>same code</u> but has its own memory and file resources. | ③ all threads share work in <u>same memory</u> and have common files, child process etc |
| ④ If one process is blocked, no other process can execute until first one is unblocked | ④ while one thread is blocked in wait, a second thread in same task can run. |
| ⑤ Multiple processes w/o using threads use more resources | ⑤ Multiple threaded processes use fewer resources |
| ⑥ Multiple processes - each process operates independently of the other. | ⑥ one thread can read/write as change another thread's data |

↓
- can be advantageous/
disadvantageous

TYPES OF THREADS

- ① User level thread - thread management & interaction done at user level (space) → no interaction w/ kernel.
→ applications build these - using thread libraries
→ fast to create & manage threads

→ issue : if a thread makes block system call → whole process shut down ⇒ all threads will be closed & running ones will be affected.

② Kernel level threads → Kernel in OS handles all threading system

→ slow to create & manage threads

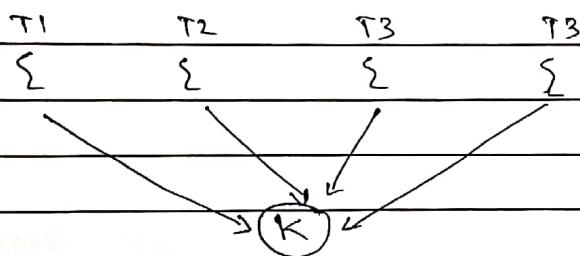
↓ ADVANTAGES

- issue in one thread, won't affect others ⇒ multithreading possible
- even works in multiprocess environment
(many CPU's kernel can shift threads b/w them)

MULTI THREADING MODELS

① Many to one

(user threads) mapped to (one kernel thread / space)

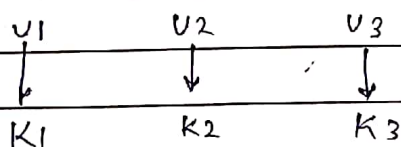


issue:

- one thread makes system call → whole system will block, because all are pointing to same kernel space, ∴ it'll block for all threads

- cannot work for multiprocesses because only 1 kernel ⇒ thread switching not possible.

② ONE to ONE

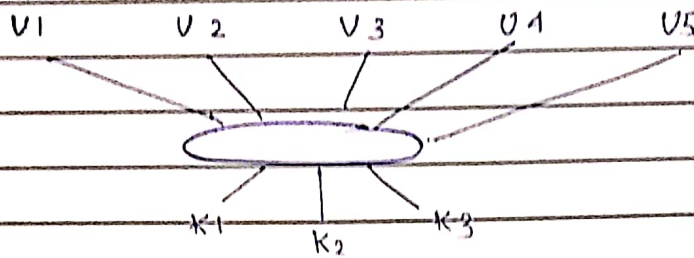


- ✓ multithreading ⇒ provides concurrency
- ✓ fault in one kernel → can point to another kernel

issue:

- kernel thread needed for every user thread (even if work is less) ⇒ lot of space required
- used in win 2000.

(3) Many to Many



- ✓ concurrency
- ✓ multi processor
- ✓ not necessarily to give each user a kernel

BOOTING PROCESS. → comp starts / reboots.

- starts → switch on button,
- ends → desktop visible.

as part of the BIOS.

① **POST** - Power on self test. (POST instructions stored in ROM)

- all devices connected w/ computer, (monitor, mouse, keyboard, internal hard drive, external hard drive etc.)
 - powers them ON & checks if in 100% working condition.
 - if any faulty device → error shown.
- this process (POST) is run by collective efforts of CPU (microprocessor) and BIOS (Basic I/O system → chip inside motherboard), and RAM. → if 3 of them working, only then POST starts.

MBR →

② BIOS finds one particular booting device (which has an OS available), so that it can load the OS onto computer's RAM.

- motherboard manufacturer sets default priority list to find booting device eg: ~~CD ROM~~ ~~OS present not auto~~

① OS available & working - eg: finds boot files on hard disk (→ command.com, MS-DOS.sys, IO.sys (I/O)) all loaded onto RAM → for windows.

② No OS → finds hard disk, CD ROM, DVD ROM, pen drive find boot files. ans installs OS.

③ Faulty / corrupt OS → BIOS searches other boot device ~~if not found~~

and checks ~~boot~~ for installation process $\rightarrow$ some files already present, then it asks to $\rightarrow$ repair / reinstall.

$\rightarrow$ POST is looking for MBR (Master Boot Record) $\rightarrow$ MBR is always at cylinder 0, Head 0, sector 1 or 001, ~~same as~~ pointing to boot sector.
Once MBR is found, control is passed to the OS.

• MBR

$\rightarrow$ ① Boot code: processes the partition table to identify which partition is bootable.

Partition table: identifies the file system on disk partition

- each entry of partition table gives the info. about where each partition will begin.

③ INIT. - last process of boot sequence.

- once kernel is loaded, find init and executes it
- init reads initialisation files that helps init to read an initial configuration script for the environment which
 - $\rightarrow$ checks the file system.
 - $\rightarrow$ sets the path etc.

④

	W	X	Y	Z	$n \rightarrow$ shared variable
wait(s)	wait(s)	wait(s)	wait(s)	wait(s)	
R(n)	R(n)	R(n)	R(n)	R(n)	
$n = n + 1$	$n = n + 1$	$n = n + 1$	$n = n - 2$	$n = n - 2$	
w(n)	w(n)	w(n)	w(n)	w(n)	
signal(s)	signal(s)	signal(s)	signal(s)	signal.	if $s = 2$, what are min and max val of n .

Race condition: value depends on order of execution.

① max $\rightarrow$ dont want Y, Z to work

$\Rightarrow W \rightarrow R(n)$, and now Y enters its CS, $\left. \begin{array}{l} \text{W, Y incs} \\ \text{then Z enters its CS} \end{array} \right\} s = 2 \Rightarrow 2 \text{ processes can enter.}$
 $\downarrow$ now
 $W \rightarrow n = n + 1$ (if read 0, before $\Rightarrow$ starts from there)

X $\dots \Rightarrow$ **MAX: 2**

② min $\rightarrow$ Z enters, W finishes, X finishes, Z resumes from read val (0)
 $\Rightarrow$ **MIN: -4**